QUE:Provide an example that defines and demonstrates the use of SQL functions.

SQL functions are used to perform operations on data, either built-in (e.g., `COUNT()`, `SUM()`, `AVG()`) or user-defined functions (UDFs).

1. Using Built-in SQL Functions

Let's assume we have a table called **`Orders`:**

OrderID	CustomerName	OrderDate	TotalAmount
1	John Doe	2024-02-01	1000
2	Jane Smith	2024-02-05	1500
3	Alice Brown	2024-02-10	1200
4	Bob White	2024-02-15	800

Example Queries with Built-in Functions

Get total orders

```
SELECT COUNT(*) AS TotalOrders FROM Orders;
```

Find the highest order amount

```
SELECT MAX(TotalAmount) AS MaxOrder FROM Orders;
```

Calculate the average order amount

```
SELECT AVG(TotalAmount) AS AvgOrderAmount FROM Orders;
```

Get total revenue

```
SELECT SUM(TotalAmount) AS TotalRevenue FROM Orders;
```

---

2. Creating and Using a User-Defined Function (UDF)

SQL also allows creating user-defined functions (UDFs) for custom calculations.

Example: Creating a Function to Calculate Discounted Price

Suppose we want to calculate a 10% discount on orders above ₹1000.

```
CREATE FUNCTION GetDiscountedPrice(@amount DECIMAL(10,2))
RETURNS DECIMAL(10,2)
AS
BEGIN
    DECLARE @discounted DECIMAL(10,2);

    IF @amount > 1000
        SET @discounted = @amount * 0.90;  -- 10% discount
    ELSE
        SET @discounted = @amount;  -- No discount

    RETURN @discounted;
END;
```

Using the Function in a Query

```
SELECT OrderID, CustomerName, TotalAmount,
       dbo.GetDiscountedPrice(TotalAmount) AS DiscountedPrice
FROM Orders;
```

Detail explanation of query
Custom Function: GetDiscountedPrice calculates a discount based on a condition.
Input Parameter: @amount (order total).
Conditional Logic: If @amount > 1000, apply a 10% discount; otherwise, return the original amount.
Return Value: The final discounted price.

Output Example

OrderID	CustomerName	TotalAmount	DiscountedPrice
1	John Doe	1000	1000
2	Jane Smith	1500	1350
3	Alice Brown	1200	1080
4	Bob White	800	800

Detail explanation of function:-

CREATE FUNCTION GetDiscountedPrice(@amount DECIMAL(10,2))

`CREATE FUNCTION` : This statement is used to define a new **User-Defined Function (UDF)** in SQL.
`GetDiscountedPrice` : The name of the function.
`(@amount DECIMAL(10,2))` : This specifies that the function takes **one input parameter** (`@amount`) of type `DECIMAL(10,2)`, meaning:

- **10** : Total digits in the number (precision).
- **2** : Digits after the decimal point (scale)

RETURNS DECIMAL(10,2)

> Specifies that the function **returns** a value of type `DECIMAL(10,2)`, meaning it will return a numeric value with up to **10 total digits**, including **2 decimal places**.

```
# AS
BEGIN
    DECLARE @discounted DECIMAL(10,2);
```

`AS BEGIN` : Marks the beginning of the function body.

`DECLARE @discounted DECIMAL(10,2);` : Declares a **local variable** named `@discounted` : to store the discounted price.

```
#  IF @amount > 1000
```

```
        SET @discounted = @amount * 0.90;  -- 10% discount
    ELSE
        SET @discounted = @amount;  -- No discount
```

- **`IF @amount > 1000`** : If the given amount is **greater than 1000**, apply a **10% discount** (`@amount * 0.90`).
- **`ELSE`** : If the amount is **1000 or less**, no discount is applied.
- `SET @discounted =` : Assigns the calculated value to the `@discounted` variable.

```
#   RETURN @discounted;
 END;
```

- **`RETURN @discounted;`** : Returns the final discounted price.
- **`END;`** : Marks the end of the function.

Example : 2

StudentID	Name	Subject	Marks	ExamDate
1	Alice	Math	85	2024-02-10
2	Bob	Math	78	2024-02-11
3	Charlie	Science	92	2024-02-12
4	David	Science	88	2024-02-13
5	Emma	Math	95	2024-02-14

---

1. Using Built-in SQL Functions

(a) Aggregate Functions
Find the total number of students

```
SELECT COUNT(*) AS TotalStudents FROM Students;
```

Find the highest and lowest marks

```
SELECT MAX(Marks) AS HighestMarks, MIN(Marks) AS LowestMarks FROM
Students;
```

Calculate the average marks in Math

```
SELECT AVG(Marks) AS AverageMathMarks FROM Students WHERE Subject =
'Math';
```

(b) String Functions

Convert student names to uppercase

```
SELECT UPPER(Name) AS UppercaseName FROM Students;
```

Get the first three letters of each student's name

```
SELECT LEFT(Name, 3) AS ShortName FROM Students;
```

(c) Date Functions

Find students who took the exam in February 2024

```
SELECT * FROM Students WHERE MONTH(ExamDate) = 2 AND YEAR(ExamDate) =
2024;
```

Get the current date

```
SELECT GETDATE() AS TodayDate;
```

---

2. Creating a User-Defined Function (UDF)

(a) Function to Check Pass/Fail

Let's define a function that checks if a student has passed (pass marks = 40).

```
CREATE FUNCTION CheckPassFail(@marks INT)
RETURNS VARCHAR(10)
```

```
AS
BEGIN
    IF @marks >= 40
        RETURN 'Pass';
    ELSE
        RETURN 'Fail';
END;
```

(b) Using the Function in a Query

```
SELECT StudentID, Name, Marks,
       dbo.CheckPassFail(Marks) AS Result
FROM Students;
```

Output Example

StudentID	Name	Marks	Result
1	Alice	85	Pass
2	Bob	78	Pass
3	Charlie	92	Pass
4	David	88	Pass
5	Emma	95	Pass